



SERVICES IN ANGULAR



codewithrutu_09

✓ What is a Service in Angular?

📌 A service in Angular is a **reusable class** that contains **business logic, data access logic, or shared functionality**.

🧠 Services help:

- Avoid code duplication
- Separate concerns (logic vs UI)
- Share data between components
- Simplify testing

✓ When Should You Use a Service?

Use services when you need to:

🔧 Share logic across components

🔄 Maintain application state

🔗 Fetch data from APIs (HTTP)

🔒 Handle authentication/authorization

🧠 Perform calculations or
transformations

📦 Services are injected using

Dependency Injection (DI)

✓ How to Create a Service

Use Angular CLI:



```
ng generate service data
```

It creates:



app.component.ts

```
@Injectable({  
  providedIn: 'root'  
})  
export class DataService {  
  getData() {  
    return ['Angular', 'React', 'Vue'];  
  }  
}
```



Injecting a Service into a Component

To use the service in a component:



app.component.ts

```
constructor(private dataService: DataService) {}  
  
ngOnInit() {  
  const items = this.dataService.getData();  
}
```



Angular's DI system provides the service automatically.



The service is singleton when provided in 'root'.



✓ Using Services for HTTP Calls

Typical HTTP service:

```
app.component.ts

@Injectable({ providedIn: 'root' })
export class ApiService {
  constructor(private http: HttpClient) {}

  getUsers(): Observable<User[]> {
    return this.http.get<User[]>('/api/users');
  }
}
```

Use it in a component:

```
app.component.ts

this.apiService.getUsers().subscribe(users => this.userList = users);
```

This keeps components clean and focused only on UI.

✓ Sharing Data Between Components

Example:

```
app.component.ts

@Injectable({ providedIn: 'root' })
export class StateService {
  private name = '';

  setName(newName: string) {
    this.name = newName;
  }

  getName(): string {
    return this.name;
  }
}
```

Now both Component A and B can read/write name via the service.

Best Practices for Angular Services

Best Practices:

- **Keep services focused on a single responsibility**
- **Do not directly manipulate the DOM (let components handle it)**
- **Use RxJS for data streams/reactivity**
- **Split services if one grows too large**
- **Use facade services to abstract complex logic**

Key Takeaways

Angular services help abstract and organize logic

Created using @Injectable and Angular CLI

Shared via Dependency Injection

Used for API calls, state management, logic reuse

Promote separation of concerns and cleaner architecture



codewithrutu_09



Like this post!



Share with your friends



Save it for later



**Tell us what you think in
comment**



codewithrutu_09



Rutuja Shinde